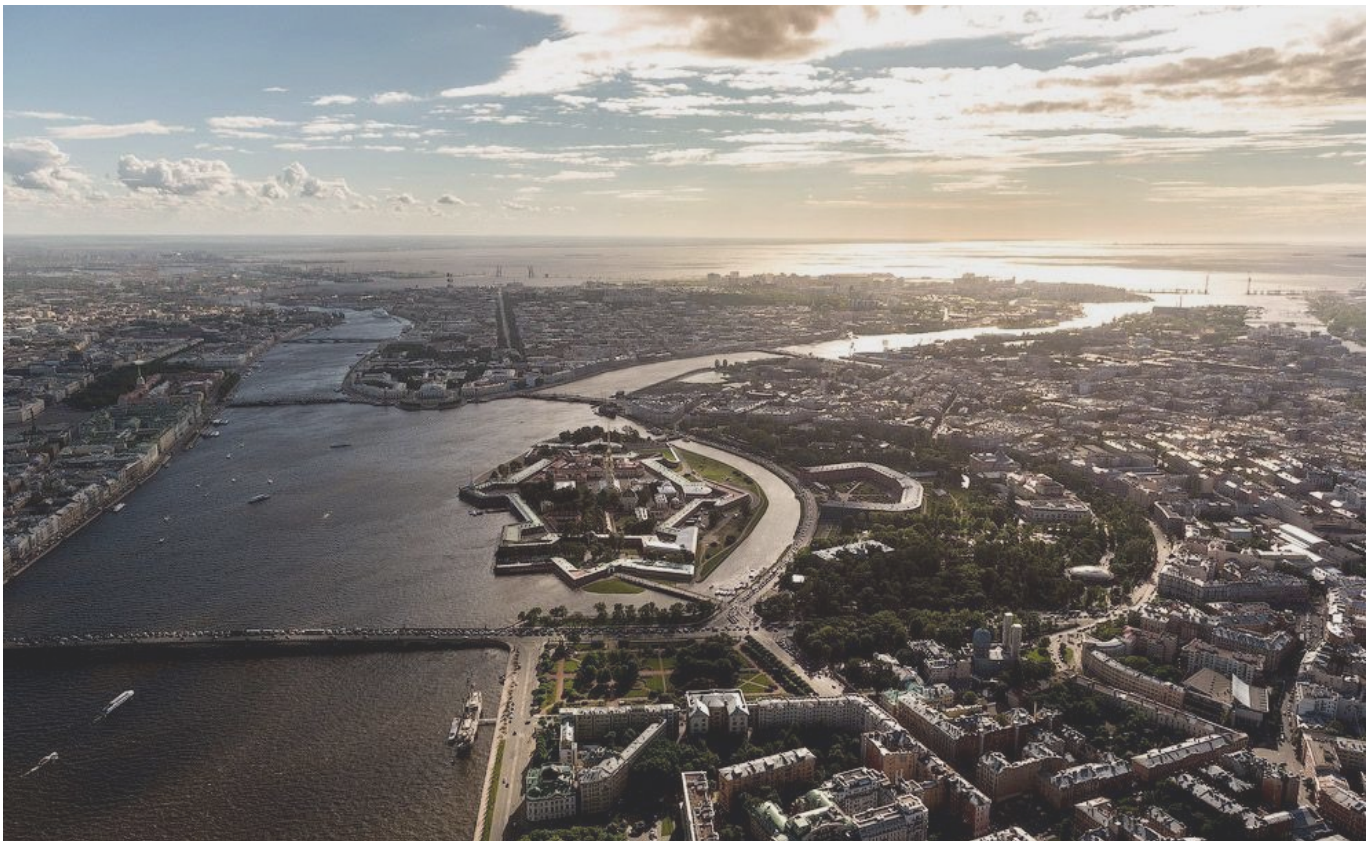
J3211, Сидельникова Дарья, 476010

ПРОЕКТИРОВАНИЕ БД

Тема 16: Поиск/покупка/аренда жилья

Лабораторная 3.

(На примере https://cian.ru/)



Санкт-Петербург
2025 г

1. Анализ запросов

Для каждого из SELECT-запросов из 2-ой лр проанализировать запросы:

- Какие столбцы используются в условиях `WHERE`, `JOIN`, `ORDER BY`, `GROUP BY`.
- Являются ли эти столбцы частыми кандидатами для поиска или сортировки.
- Какой тип индекса (обычный B-дерево, составной и т.д.) мог бы ускорить выполнение этого запроса.

Основные типы:

1.B-tree
– Стандартный, по умолчанию.
– Хорош для =, >, <, BETWEEN, ORDER BY. Универсальный.
2.Hash
– Только для =.
– Быстрее B-tree в чистых сравнениях, но бесполезен для сортировок и диапазонов.
3.GiST
– “Гибкое дерево” для сложных типов.
– Подходит для геоданных, диапазонов, похожести и т.п.
4.GIN
– Инвертированный индекс.
– Хорош для массивов, JSON и полнотекстового поиска (много значений в одном поле).
5.BRIN
– Индекс по диапазонам блоков, а не по строкам.
– Полезен на очень больших таблицах, где данные лежат примерно по порядку (например, по дате).
6.Обычный индекс по одному столбцу
– Ускоряет поиск по одному полю.
7.Составной индекс
– Несколько столбцов в одном индексе.
– Удобен, когда в запросе сразу несколько условий/сортировок.
8.Уникальный индекс
– Как обычный, но ещё запрещает дубликаты значений.

Запросы:

Топ-5 самых дорогих объявлений на продажу

```
– select l.listing_id, p.address, l.price, pt.type_name, d.district_name
from listings l
join properties p on l.property_id = p.property_id
join property_types pt on p.property_type_id = pt.type_id
join districts d on p.district_id = d.district_id
where l.is_active = true
  and l.deal_type_id = (select deal_type_id from deal_types where deal_type_name =
'sale')
order by l.price desc
limit 5;
```

– JOIN:

• l.property_id = p.property_id

• p.property_type_id = pt.type_id

• p.district_id = d.district_id

– WHERE:

• l.is_active

• l.deal_type_id

• deal_types.deal_type_name (в подзапросе)

– ORDER BY:

• l.price (по убыванию)

Очень частые:

– deal_type_id и is_active —"только продажа/аренда", "только активные объявления".

Частые:

– price — "дороже/дешевле сначала".

– deal_type_name — поиск по названию типа сделки

Я думаю составной B-tree индекс на (deal_type_id, is_active, price DESC) в таблице listings и обычный B-tree индекс на deal_types(deal_type_name) могли бы ускорить выполнение этого запроса, потому что первый сразу найдёт активные объявления нужного типа в порядке по цене, а второй быстро подберёт id типа сделки по его названию

Статистика по районам;

```
select d.district_name,count(l.listing_id) as total_listings, avg(l.price) as
avg_price, max(l.price) as max_price
from districts d
left join properties p on d.district_id = p.district_id
left join listings l on p.property_id = l.property_id and l.is_active = true
group by d.district_id, d.district_name
```

```
having count(l.listing_id) > 0
order by total_listings desc;
```

– JOIN:
• d.district_id = p.district_id
• p.property_id = l.property_id
• l.is_active

– GROUP BY:
• d.district_id
• d.district_name

– HAVING:
• count(l.listing_id)

– ORDER BY:
• total_listings

Частые ::
– district_id в properties — фильтр “по району”.
– property_id в listings — связь “объявление - объект”.
– is_active в listings

Я думаю обычный B-tree индекс на properties(district_id) и составной B-tree индекс на listings(property_id, is_active), потому что первый быстро отдаёт все объекты по району, а второй быстро находит только активные объявления для каждого объекта.

Объекты с площадью выше средней по району

```
select p.property_id, p.address, p.total_area, d.district_name,
    (select avg(p2.total_area)
     from properties p2
     where p2.district_id = p.district_id) as avg_area_in_district
from properties p
join districts d on p.district_id = d.district_id
where p.total_area > (select avg(p2.total_area)
                      from properties p2
                      where p2.district_id = p.district_id)
order by d.district_name, p.total_area desc;
```

Столбцы в условиях
– JOIN:
• p.district_id = d.district_id

– WHERE

• p.total_area

– ORDER BY:
• d.district_name
• p.total_area

Частота использования
– district_id — разрез по району
– total_area — площадь больше X

Думаю составной B-tree индекс на properties(district_id, total_area) мог бы ускорить выполнение этого запроса, потому что он позволяет быстро работать с объектами внутри одного района и сразу учитывать их площади для расчёта средней и сравнения "больше средней".

Рейтинг риелторов

```
with agent_stats as (select u.user_id, u.first_name || ' ' || u.last_name as
agent_name, count(l.listing_id) as active_listings, avg(l.price) as avg_price
    from users u
    join properties p on u.user_id = p.owner_id
    join listings l on p.property_id = l.property_id
    where l.is_active = true
    group by u.user_id, agent_name
)
select agent_name, active_listings,round(avg_price, 2) as avg_price,
    case
        when active_listings >= 5 then 'Топ'
        when active_listings >= 2 then 'Средний'
        else 'Начинающий'
    end as rating
from agent_stats
order by active_listings desc, avg_price desc;
```

– JOIN:
• u.user_id = p.owner_id
• p.property_id = l.property_id

– WHERE:
• l.is_active

– GROUP BY:
• u.user_id
• agent_name
•
– ORDER BY:
• active_listings
• avg_price

Частота
– owner_id в properties — все объекты этого пользователя/агента
– property_id в listings — связь “объект - объявление”
– is_active

Я думаю обычный B-tree индекс на properties(owner_id) и составной B-tree индекс на listings(property_id, is_active), потому что первый быстро находит все объекты конкретного риелтора, а второй быстро подбирает только активные объявления по этим объектам

Объекты с полным набором удобств

```
select distinct p.property_id, p.address, p.total_area, pt.type_name,
string_agg(a.amenity_name, ', ') as amenities
from properties p
join property_types pt on p.property_type_id = pt.type_id
join property_amenities pa on p.property_id = pa.property_id
join amenities a on pa.amenity_id = a.amenity_id
where a.amenity_name in ('парковка', 'лифт', 'консьерж')
  and p.total_area > 50
  and p.rooms_count >= 2
group by p.property_id, p.address, p.total_area, pt.type_name
having count(distinct a.amenity_name) = 3
order by p.total_area desc;
```

Столбцы в условиях
– JOIN:
• p.property_type_id = pt.type_id
• p.property_id = pa.property_id
• pa.amenity_id = a.amenity_id

– WHERE:
• a.amenity_name
• p.total_area
• p.rooms_count

– GROUP BY:
• p.property_id
• p.address
• p.total_area
• pt.type_name

– HAVING:
• count

– ORDER BY:
• p.total_are

•
• Частота
– amenity_name — нужна парковка и лифт
– total_area и rooms_count — площадь не меньше X комнат не меньше N

Я думаю обычный (уникальный) B-tree индекс на amenities(amenity_name) и составной B-tree индекс на properties(total_area, rooms_count) могли бы ускорить выполнение этого запроса, потому что первый быстро находит нужные удобства по имени, а второй сразу отфильтровывает объекты с подходящей площадью и количеством комнат и помогает сортировать по площади.

2. Создание индексов

Создайте индексы на нужные поля.

Для каждого индекса укажите:
- Sql запрос для создания
- почему выбран именно этот тип индекса

SQL-код

```
CREATE UNIQUE INDEX IF NOT EXISTS idx_deal_types_name
ON deal_types (deal_type_name);
```

Я выбрала уникальный B-tree индекс по столбцу deal_type_name, потому что названия типов сделок не должны повторяться, и по ним удобно находить deal_type_id в подзапросах. Это защищает от дублей, B-tree хорошо подходит для быстрого поиска по равенству.

```
CREATE INDEX IF NOT EXISTS idx_listings_deal_active_price
ON listings (deal_type_id, is_active, price DESC);
```

Тут сделала составной B-tree индекс по (deal_type_id, is_active, price) в таблице listings, потому что запросы сначала фильтруют объявления по типу сделки и активности, а потом сортируют по цене. Индекс позволяет сразу найти все активные объявления нужного типа сделки и уже в индексе пройтись по ним в порядке убывания цены, без лишней сортировки таблицы.

```
CREATE INDEX IF NOT EXISTS idx_listings_property_active
ON listings (property_id, is_active);
```

Тут B-tree индекс по (property_id, is_active), потому что часто нужно искать активные объявления конкретного объекта. В этом индексе все строки для одного property_id лежат рядом, и база быстро находит среди них только те, у которых is_active = true.

```
CREATE INDEX IF NOT EXISTS idx_properties_district_total_area
ON properties (district_id, total_area);
```

Сделала B-tree индекс по (district_id, total_area), потому что в запросах сравнивается площадь объекта со средней площадью по району. Такой индекс группирует объекты по району и сразу хранит их площади, поэтому быстрее считать среднее значение и проверять условие “площадь больше средней в этом районе”.

```
CREATE INDEX IF NOT EXISTS idx_properties_area_rooms
ON properties (total_area, rooms_count);
```

Выбрала составной B-tree индекс по (total_area, rooms_count), потому что в запросах фильтруют объекты по минимальной площади и количеству комнат и сортируют по площади. Этот индекс помогает быстро откинуть все квартиры с маленькой площадью или недостаточным числом комнат и одновременно использовать total_area для сортировки.

```
CREATE UNIQUE INDEX IF NOT EXISTS idx_amenities_name
ON amenities (amenity_name);
```

Взяла тут уникальный B-tree индекс по amenity_name, потому что названия удобств должны быть уникальными (нельзя иметь две разные строки “парковка”) и по ним часто делают фильтр IN в запросах. Уникальный индекс и ускоряет поиск по имени, и не даёт случайно добавить одинаковое удобство второй раз.

3. Анализ запросов на модификацию данных

Проанализируйте созданные индексы и написанные ранее запросы для модификации данных. Не помешают ли индексы?

Анализ есть выше.
Я считаю, что созданные мной индексы не мешают запросам на модификацию данных:
• при INSERT и DELETE в таблицах listings, properties, deal_types и amenities операции становятся чуть тяжелее, потому что базе нужно обновлять индексы
• при UPDATE, если меняются поля, которые входят в индекс (например price или total_area), запись в индексе тоже переставляется, и это немного удорожает обновление

Критичных проблем это не создаёт, модифицирующие запросы по-прежнему работают нормально, а индексы в основном только помогают — ускоряют SELECT-запросы из ЛР №2

4. Дополнительные вопросы

1.Почему не стоит создавать индекс по каждому столбцу?

Я считаю, что нельзя делать индекс на каждый столбец, потому что каждый индекс занимает место и замедляет INSERT/UPDATE/DELETE. Часть столбцов почти не участвует в запросах, и индексы на них просто будут висеть без пользы.

2.В каких случаях индекс может ухудшить запрос?

Думаю, индекс может ухудшить запрос, когда таблица маленькая и быстрее прочитать её целиком, чем идти по индексу. Ещё плохо, когда условие выбирает почти все строки (низкая селективность) или когда индексов слишком много и из-за них сильно тормозят операции записи.

3.Что такое селективность столбца, и как она влияет на полезность индекса?

Я понимаю селективность как долю строк которое выбирает условие по этому столбцу. Чем селективность выше (то есть возвращается мало строк), тем полезнее индекс, потому что он реально сокращает объём читаемых данных. Если селективность низкая и подходит почти вся таблица, индекс обычно мало помогает.

4.Что такое кардинальность столбца и почему ее нужно учитывать?

Я считаю, что кардинальность столбца — это количество разных значений в нём. При высокой кардинальности (много уникальных значений) индекс обычно полезен, а при низкой (пара-тройка вариантов) индекс по одному этому полю почти не даёт выигрыша. Поэтому кардинальность нужно учитывать, чтобы не создавать бессмысленные индексы на “одно и то же” значение.